

Laporan Tugas Kecil 2

IF2211 STRATEGI ALGORITMA
SEMESTER II TAHUN 2025/2026



Voxelization Objek 3D menggunakan Octree dengan Algoritma Divide and Conquer

Disusun Oleh:

13524090 Nashiruddin Akram

13524093 Reinsen Silitonga

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Maret 2026

Contents

1	Algoritma Divide and Conquer	2
1.1	Gambaran Umum	2
1.2	Langkah-langkah Algoritma	2
1.2.1	Membangun Root Bounding Box	2
1.2.2	Divide: Membagi Kubus Menjadi 8 Oktant	2
1.2.3	Memeriksa Irisan Permukaan	2
1.2.4	Conquer: Rekursi ke Oktant yang Relevan	3
1.2.5	Pembentukan Voxel sebagai Base Case	3
1.3	Pseudocode	3
1.4	Kompleksitas Waktu	3
2	Implementasi	4
2.1	Struktur File	4
2.2	Modul Program	4
2.3	Validasi Input	4
2.4	Format Output	5
3	Eksperimen dan Hasil	5
3.1	Hasil 1: Kubus pada depth 4	5
3.2	Hasil 2: Line pada depth 3	7
3.3	Hasil 3: Pumpkin pada depth 5	9
3.4	Hasil 4: Pumpkin pada depth 7	11
3.5	Hasil 5: Cow pada depth 6	14
3.6	Hasil 6: Cow pada depth 8	16
3.7	Hasil 7: Teapot pada depth 6	19
3.8	Hasil 8: Input Tidak Valid	21
4	Bonus Visualisasi	22
4.1	Alur Rendering dan Interaksi	22
4.1.1	Membaca Model dan Menyiapkan Data Geometri	22
4.1.2	Normalisasi Model terhadap <i>Bounding Box</i>	22
4.1.3	Transformasi Ruang 3D ke Layar 2D	22
4.1.4	Penyaringan, Pengurutan, dan Pewarnaan Segitiga	22
4.1.5	Interaksi Pengguna	22
4.2	Perbandingan Visualisasi Web dan Visualizer C++	22
4.2.1	Contoh 1: Objek Cow (depth 6)	23
4.2.2	Contoh 2: Objek Pumpkin (depth 5)	23
4.2.3	Contoh 3: Objek Teapot (depth 7)	24
4.3	Perbandingan Umum	24
5	Lampiran	25
6	Pernyataan Tidak Melakukan Kecurangan	26

1 Algoritma Divide and Conquer

1.1 Gambaran Umum

Algoritma melakukan *surface voxelization* dengan pendekatan *divide and conquer* berbasis Octree. Model masukan dibaca sebagai kumpulan segitiga yang tersusun dari titik-titik OBJ. Ruang 3D yang memuat model kemudian dibungkus oleh sebuah kubus akar. Kubus ini menjadi node awal pada struktur Octree.

Pada setiap langkah rekursi, satu node kubus dievaluasi apakah masih berhubungan dengan permukaan model. Jika tidak berhubungan, node tersebut dipangkas. Jika masih berhubungan dan kedalaman belum mencapai batas, node dibagi menjadi delapan subkubus berukuran sama. Jika kedalaman sudah mencapai batas, node tersebut dicetak sebagai satu voxel kubus pada berkas keluaran.

Dengan strategi ini, program tidak menelusuri seluruh ruang secara merata. Program hanya melanjutkan rekursi pada area yang dekat dengan permukaan objek, sehingga waktu komputasi lebih efisien dibanding pembagian seragam tanpa *pruning*.

1.2 Langkah-langkah Algoritma

1.2.1 Membangun Root Bounding Box

Sebelum rekursi dimulai, parser membaca semua baris **v** sebagai titik dan semua baris **f** sebagai bidang segitiga. Pada saat membaca titik, program sekaligus menghitung nilai minimum dan maksimum untuk sumbu x , y , dan z . Hasilnya adalah *bounding box* terkecil yang memuat seluruh model.

Karena Octree bekerja paling konsisten pada node berbentuk kubus, *bounding box* awal ditransformasi menjadi kubus dengan panjang sisi sama pada semua sumbu. Ukuran setengah sisi kubus akar dihitung dengan rumus berikut:

$$\text{half} = \max(\Delta x, \Delta y, \Delta z) \times 0.5 \times (1 + 10^{-4})$$

Faktor 10^{-4} memberi *padding* kecil agar kasus titik yang tepat berada di batas kubus tidak mudah terpengaruh galat numerik.

1.2.2 Divide: Membagi Kubus Menjadi 8 Oktant

Untuk setiap node yang relevan dan belum mencapai kedalaman maksimum, kubus dibagi di titik tengah ketiga sumbu:

$$m_x = \frac{\min_x + \max_x}{2}, \quad m_y = \frac{\min_y + \max_y}{2}, \quad m_z = \frac{\min_z + \max_z}{2}$$

Delapan oktant dibentuk dari kombinasi posisi kiri atau kanan, bawah atau atas, serta depan atau belakang. Setiap oktant memiliki panjang sisi setengah dari node induk. Mekanisme ini menghasilkan partisi ruang hierarkis yang tetap seragam pada setiap level.

1.2.3 Memeriksa Irisan Permukaan

Fungsi `buildOctree` memanggil fungsi deteksi irisan pada modul Octree untuk menentukan apakah sebuah node masih memiliki keterkaitan dengan permukaan model. Pada implementasi ini, uji dilakukan dengan pendekatan titik representatif untuk setiap bidang segitiga, yaitu tiga titik sudut, tiga titik tengah sisi, dan satu titik pusat segitiga.

Sebuah bidang dianggap relevan terhadap node jika minimal satu titik representatifnya berada di dalam *bounding box* node. Jika tidak ada bidang yang relevan, node dinyatakan kosong dan dihentikan dari rekursi.

Langkah ini adalah inti efisiensi algoritma. Ruang kosong tidak diwariskan ke level berikutnya, sehingga jumlah node yang benar-benar diproses dapat berkurang secara signifikan.

1.2.4 Conquer: Rekursi ke Oktant yang Relevan

Untuk setiap oktant yang lolos filter, `buildOctree` dipanggil kembali dengan *bounding box* oktant sebagai domain baru. Nilai kedalaman dinaikkan dari d menjadi $d + 1$. Proses ini berlanjut sampai salah satu dari dua kondisi terpenuhi, yaitu node dipangkas karena kosong, atau node mencapai kedalaman maksimum.

1.2.5 Pembentukan Voxel sebagai Base Case

Algoritma tidak akan terus membelah ruang selamanya. Kami membuat algoritma ini dengan memiliki dua kondisi berhenti (base case) yang mengontrol agar pembagian ini efisien dan menghasilkan output:

1. **Pemangkasan Ruang Kosong (Pruning):** Sebelum membelah sebuah kotak, algoritma akan mengecek apakah kotak tersebut memuat atau bersinggungan dengan minimal satu bagian (face) dari model 3D aslinya menggunakan fungsi `punyaFace`. Jika kotak tersebut terbukti kosong (!ok), maka pencarian di dalam cabang atau kotak tersebut dihentikan sepenuhnya. Ini mencegah algoritma membuang-buang waktu memproses ruang udara kosong.
2. **Mencapai Target Kedalaman Maksimal:** Jika pemotongan sudah mencapai target jumlah kedalaman yang diinputkan oleh pengguna ($d = md$), algoritma menganggap bahwa ruang kecil itu mewakili satu blok voxel yang valid. Pada tahap ini, algoritma mencetak bangun kubus tersebut langsung ke dalam file output `.obj` melalui fungsi `printObjFile`, lalu proses membelah dihentikan.

1.3 Pseudocode

Algorithm 1 `buildOctree`

```
1: procedure BUILD OCTREE(box, bidang, depth, maxDepth)
2:   catat statistik node pada depth
3:   if tidak ada bidang yang menyentuh box then
4:     catat statistik prune
5:     return
6:   end if
7:   if  $depth = maxDepth$  then
8:     tulis 1 voxel kubus ke file OBJ
9:     return
10:  end if
11:   $mid \leftarrow center(box)$ 
12:  for setiap oktant  $o$  do
13:    BUILD OCTREE( $o$ , bidang,  $depth + 1$ , maxDepth)
14:  end for
15: end procedure
```

1.4 Kompleksitas Waktu

Analisis berikut hanya membahas kompleksitas waktu dalam notasi O . Misalkan d adalah kedalaman maksimum Octree dan N adalah jumlah bidang segitiga pada model masukan. Pada setiap node, proses filter dapat memeriksa hingga N bidang. Pada level ke- k , jumlah node maksimum adalah 8^k . Oleh karena itu, batas atas waktu total dapat ditulis sebagai:

$$T(d) = O\left(\sum_{k=0}^{d-1} 8^k \cdot N\right) = O(8^d \cdot N)$$

2 Implementasi

2.1 Struktur File

- `bin/voxelizer.exe`: berkas executable hasil kompilasi.
- `bin/viewer.exe`: berkas executable untuk visualisasi model OBJ.
- `doc/Laporan_Tucil2.pdf`: keluaran dokumen laporan.
- `src/geometry.hpp` dan `src/geometry.cpp`: struktur geometri dasar serta perhitungan bounding box.
- `src/parser.hpp` dan `src/parser.cpp`: parser OBJ dan pembentukan path keluaran.
- `src/octree.hpp` dan `src/octree.cpp`: rekursi octree, deteksi irisan, dan emisi voxel.
- `src/main.cpp`: antarmuka CLI, timer, dan laporan statistik.
- `src/viewer.hpp` dan `src/viewer.cpp`: modul visualisasi untuk menampilkan model hasil voxelization.
- `test/`: kumpulan data uji.
- `makefile`: aturan kompilasi dan pembersihan build.

2.2 Modul Program

Table 1: Modul-modul program

Modul	Fungsi
<code>geometry</code>	Struktur <code>Vec3</code> , <code>Box/BoundingBox</code> , dan <code>Triangle</code> ; serta fungsi geometri dan bounding box
<code>parser</code>	Membaca file <code>.obj</code> dengan validasi lengkap, menghasilkan path output
<code>octree</code>	Deteksi irisan, emisi voxel, serta <code>buildOctree</code> secara rekursif
<code>main</code>	Parsing argumen CLI, orkestrasi alur, timer proses, laporan terminal
<code>viewer</code>	Menampilkan model OBJ hasil voxelization secara interaktif

2.3 Validasi Input

Program melakukan validasi menyeluruh pada file `.obj`:

- Menolak titik dengan format tidak valid (kurang dari 3 koordinat).
- Menolak bidang non segitiga (lebih dari 3 titik).
- Menolak indeks bidang yang melebihi jumlah titik yang tersedia.
- Menolak indeks negatif.
- Prefix selain `v` dan `f` diabaikan (`vn`, `vt`, `#`, dll.).

2.4 Format Output

Setiap voxel diekspor sebagai kubus dengan:

- **8 titik** di sudut-sudut kubus.
- **12 bidang segitiga** (2 segitiga per sisi $\times$ 6 sisi), semua indeks positif.
- Ukuran kubus **seragam** (semua sisi sama panjang).

3 Eksperimen dan Hasil

3.1 Hasil 1: Kubus pada depth 4

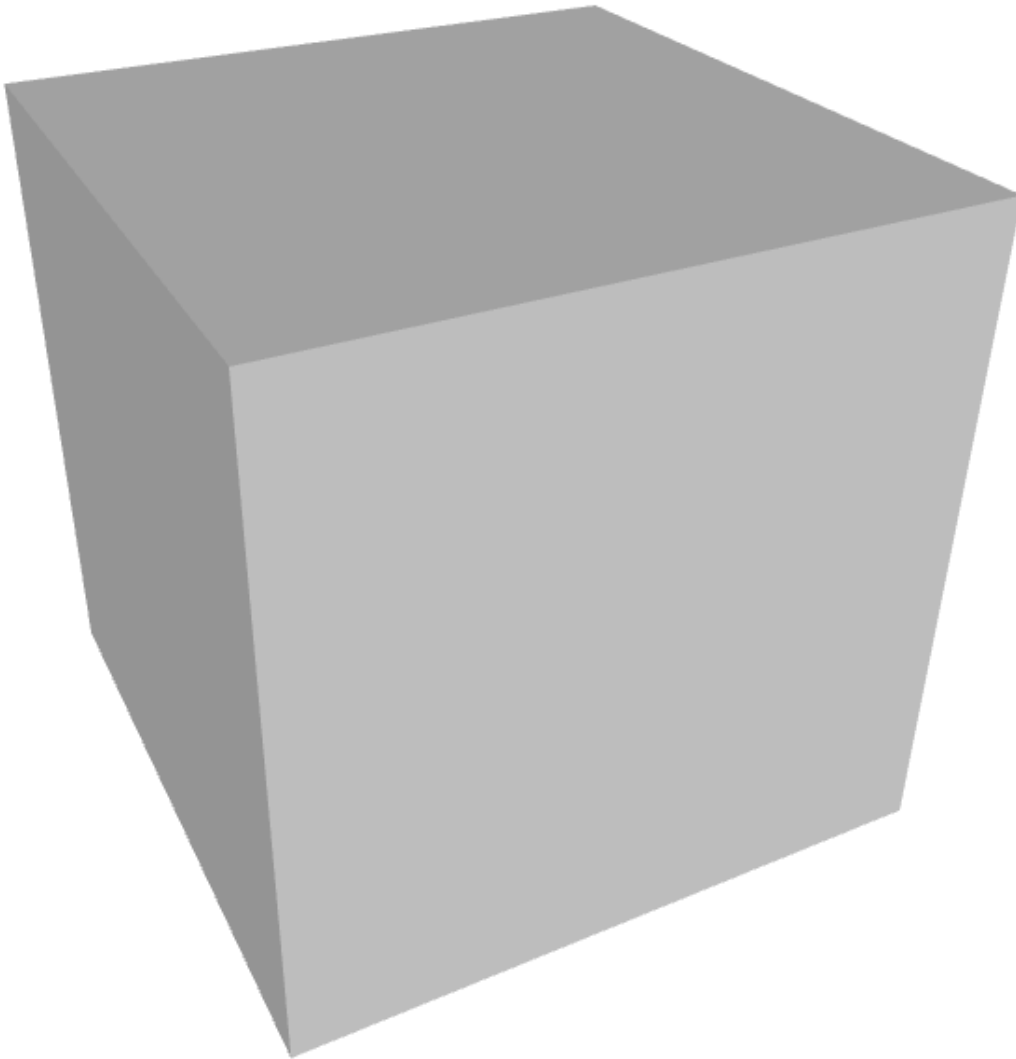


Figure 1: Gambar input model kubus

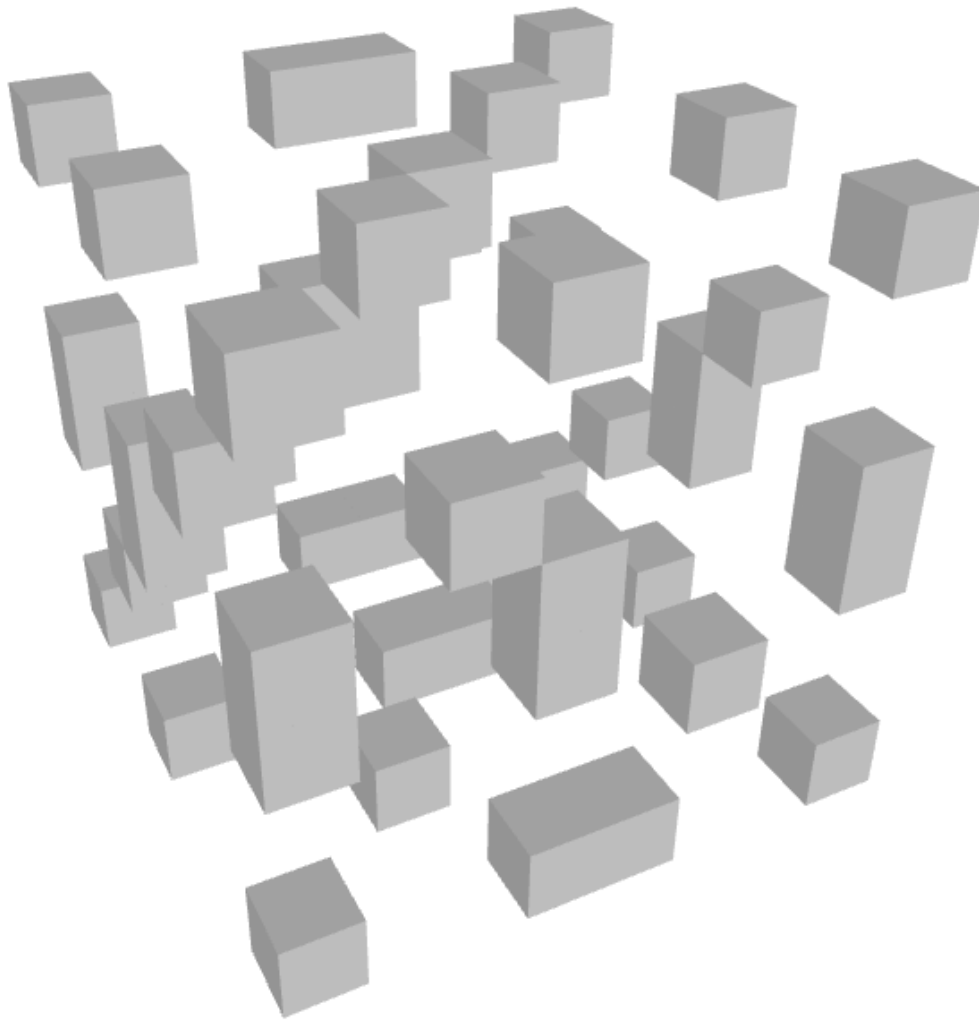


Figure 2: Gambar output voxel kubus pada depth 4

Listing 1: Output terminal kubus pada depth 4

```
$ ./bin/voxelizer.exe test/cube.obj 4
Berhasil membaca 8 vertex dan 12 face.
Membangun octree pada kedalaman 4...

Banyaknya voxel yang terbentuk      : 52
Banyaknya vertex yang terbentuk     : 416
Banyaknya faces yang terbentuk      : 624
Kedalaman octree                    : 4

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 360
```

Statistik node yang tidak perlu ditelusuri:

1 : 0
2 : 0
3 : 19
4 : 308

Lama waktu program berjalan : 0.0018 s

Path file .obj disimpan : `test/cube-voxelized-4.obj`

3.2 Hasil 2: Line pada depth 3

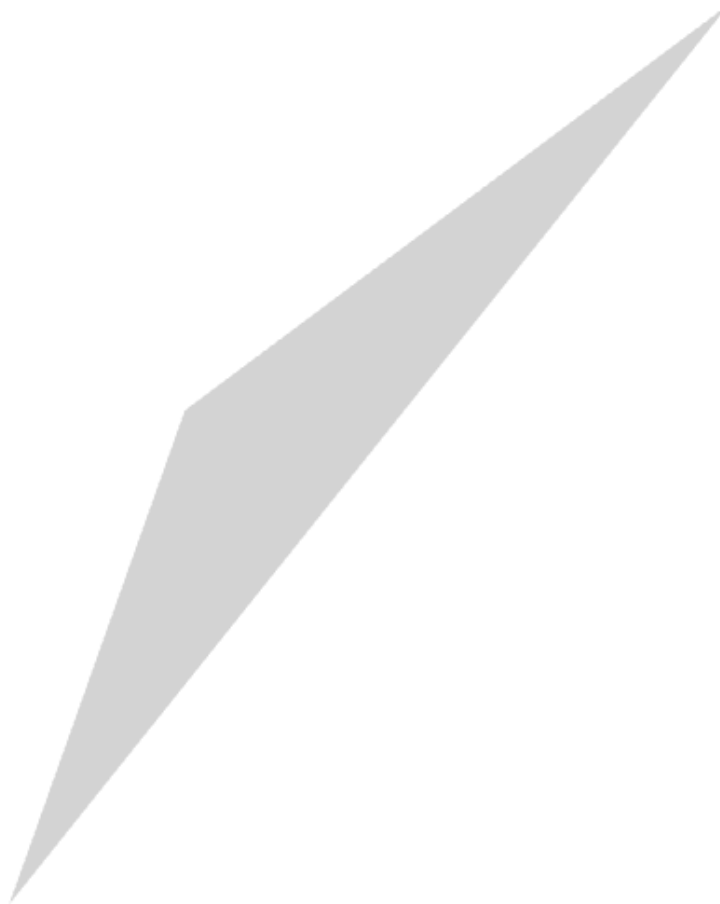


Figure 3: Gambar input model line

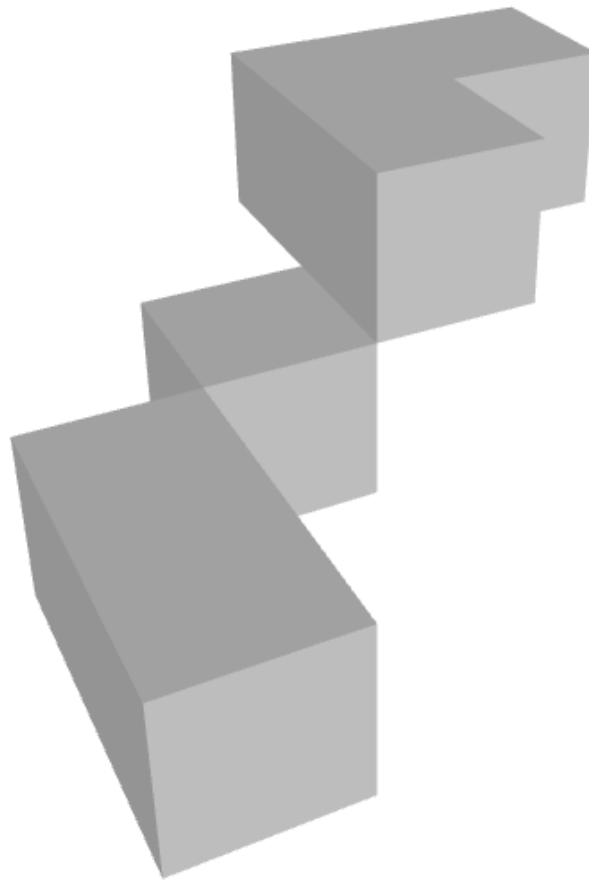


Figure 4: Gambar output voxel line pada depth 3

Listing 2: Output terminal line pada depth 3

```
./bin/voxelizer.exe test/line.obj 3
Berhasil membaca 3 vertex dan 1 face.
Membangun octree pada kedalaman 3...

Banyaknya voxel yang terbentuk      : 6
Banyaknya vertex yang terbentuk     : 48
Banyaknya faces yang terbentuk      : 72
Kedalaman octree                    : 3

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 24

Statistik node yang tidak perlu ditelusuri:
```

```
1 : 0  
2 : 5  
3 : 18
```

```
Lama waktu program berjalan      : 0.0002 s  
Path file .obj disimpan         : test/line-voxelized-3.obj
```

3.3 Hasil 3: Pumpkin pada depth 5



Figure 5: Gambar input model pumpkin

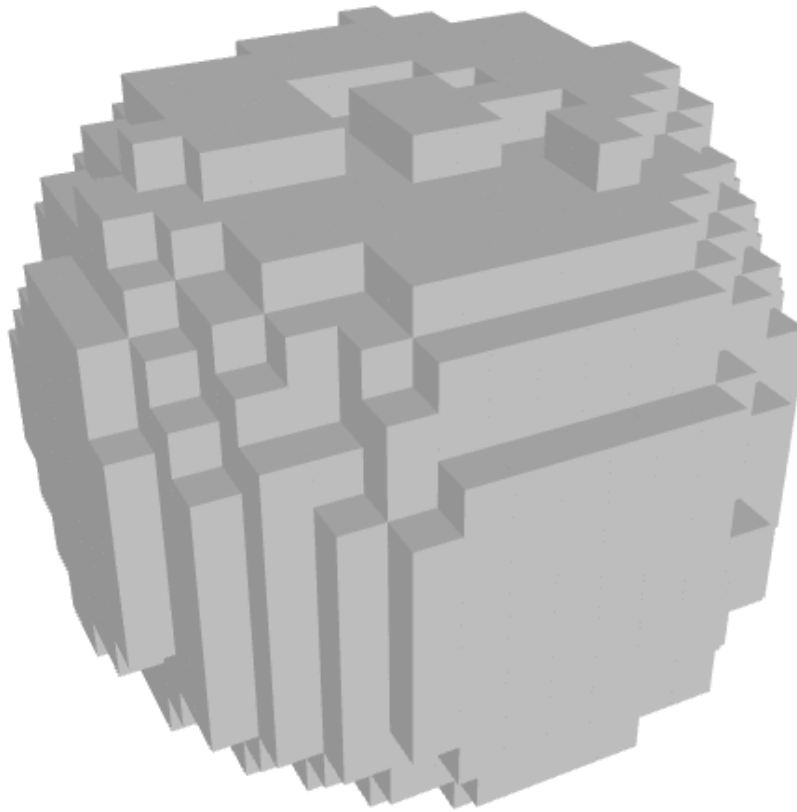


Figure 6: Gambar output voxel pumpkin pada depth 5

Listing 3: Output terminal pumpkin pada depth 5

```
$ ./bin/voxelizer.exe test/pumpkin.obj 5
Berhasil membaca 5002 vertex dan 10000 face.
Membangun octree pada kedalaman 5...

Banyaknya voxel yang terbentuk      : 1049
Banyaknya vertex yang terbentuk     : 8392
Banyaknya faces yang terbentuk      : 12588
Kedalaman octree                    : 5

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 448
5 : 2144
```

Statistik node yang tidak perlu ditelusuri:

1 : 0
2 : 0
3 : 8
4 : 180
5 : 1095

Lama waktu program berjalan : 0.3391 s

Path file .obj disimpan : test/pumpkin-voxelized-5.obj

3.4 Hasil 4: Pumpkin pada depth 7



Figure 7: Gambar input model pumpkin

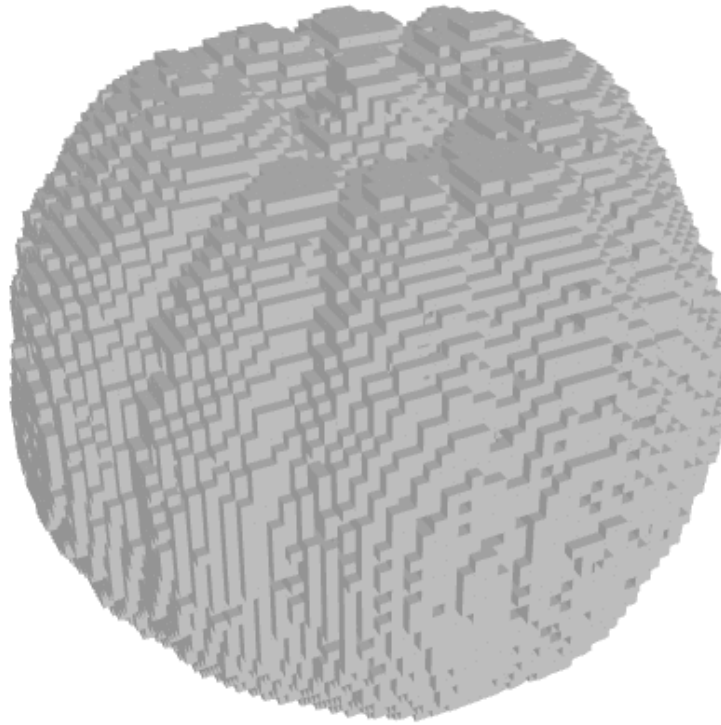


Figure 8: Gambar output voxel pumpkin pada depth 7

Listing 4: Output terminal pumpkin pada depth 7

```
$ ./bin/voxelizer.exe test/pumpkin.obj 7
Berhasil membaca 5002 vertex dan 10000 face.
Membangun octree pada kedalaman 7...

Banyaknya voxel yang terbentuk      : 13270
Banyaknya vertex yang terbentuk     : 106160
Banyaknya faces yang terbentuk      : 159240
Kedalaman octree                    : 7

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 448
5 : 2144
```

```
6 : 8392
7 : 32368
```

```
Statistik node yang tidak perlu ditelusuri:
```

```
1 : 0
2 : 0
3 : 8
4 : 180
5 : 1095
6 : 4346
7 : 19098
```

```
Lama waktu program berjalan      : 5.3597 s
```

```
Path file .obj disimpan         : test/pumpkin-voxelized-7.obj
```

3.5 Hasil 5: Cow pada depth 6

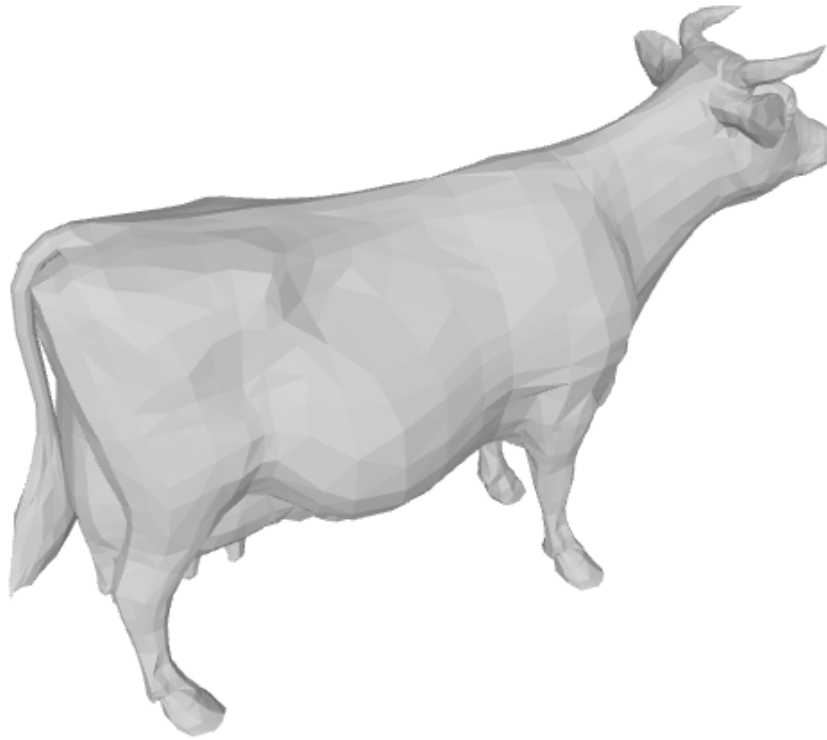


Figure 9: Gambar input model cow

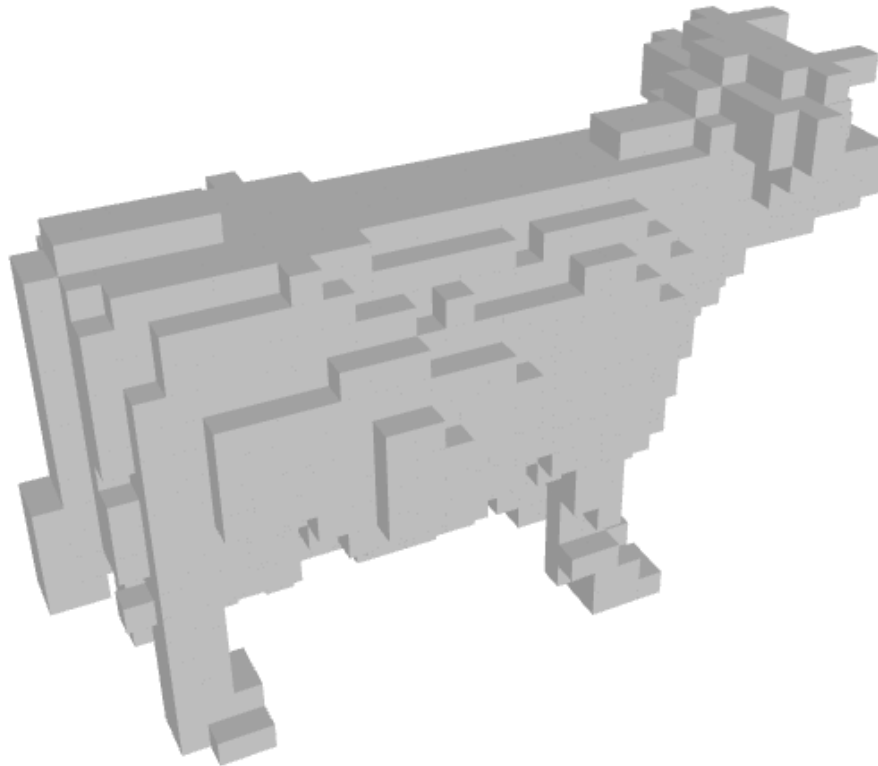


Figure 10: Gambar output voxel cow pada depth 6

Listing 5: Output terminal cow pada depth 6

```
$ ./bin/voxelizer.exe test/cow.obj 6
Berhasil membaca 4583 vertex dan 5804 face.
Membangun octree pada kedalaman 6...

Banyaknya voxel yang terbentuk      : 1248
Banyaknya vertex yang terbentuk     : 9984
Banyaknya faces yang terbentuk      : 14976
Kedalaman octree                    : 6

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 160
5 : 688
```



```
6 : 2608
```

```
Statistik node yang tidak perlu ditelusuri:
```

```
1 : 0
```

```
2 : 0
```

```
3 : 44
```

```
4 : 74
```

```
5 : 362
```

```
6 : 1360
```

```
Lama waktu program berjalan      : 0.1927 s
```

```
Path file .obj disimpan         : test/cow-voxelized-6.obj
```

3.6 Hasil 6: Cow pada depth 8

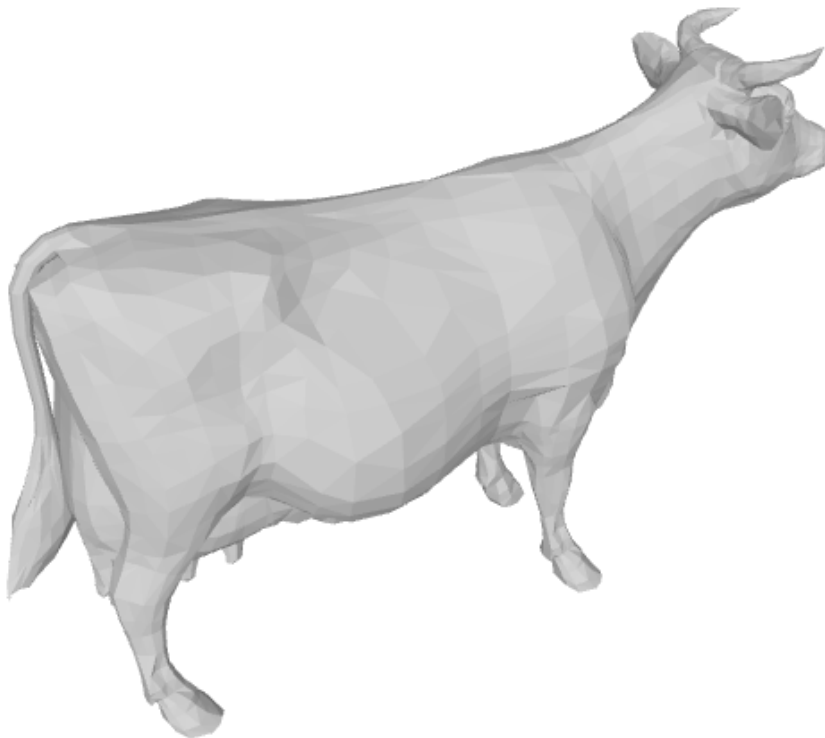


Figure 11: Gambar input model cow

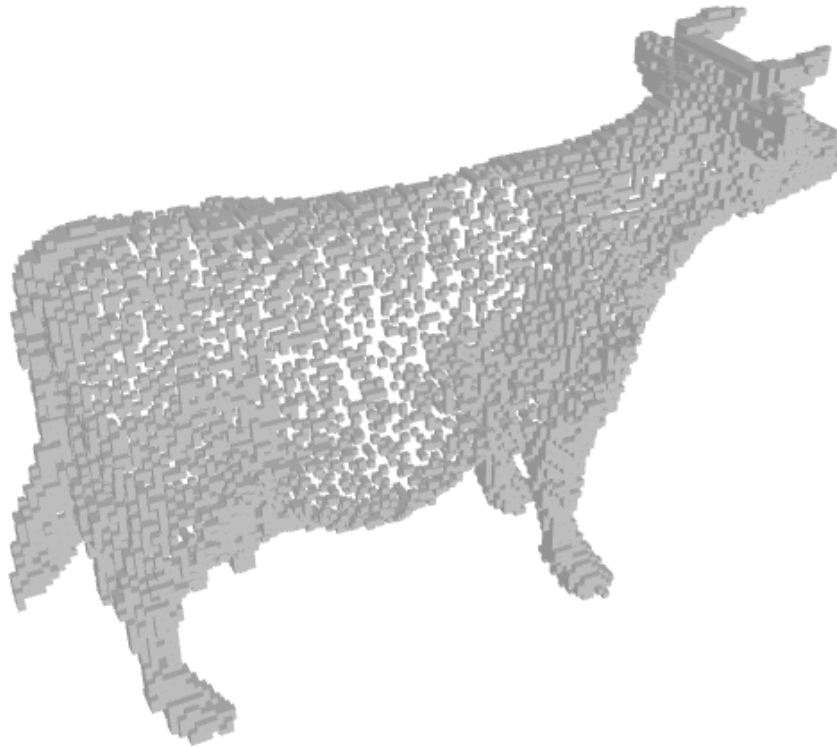


Figure 12: Gambar output voxel cow pada depth 8

Listing 6: Output terminal cow pada depth 8

```
$ ./bin/voxelizer.exe test/cow.obj 8
Berhasil membaca 4583 vertex dan 5804 face.
Membangun octree pada kedalaman 8...

Banyaknya voxel yang terbentuk      : 10205
Banyaknya vertex yang terbentuk     : 81640
Banyaknya faces yang terbentuk      : 122460
Kedalaman octree                    : 8

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 160
5 : 688
```



```
6 : 2608
7 : 9984
8 : 33896
```

Statistik node yang tidak perlu ditelusuri:

```
1 : 0
2 : 0
3 : 44
4 : 74
5 : 362
6 : 1360
7 : 5747
8 : 23691
```

```
Lama waktu program berjalan      : 2.5495 s
Path file .obj disimpan          : test/cow-voxelized-8.obj
```

3.7 Hasil 7: Teapot pada depth 6



Figure 13: Gambar input model teapot

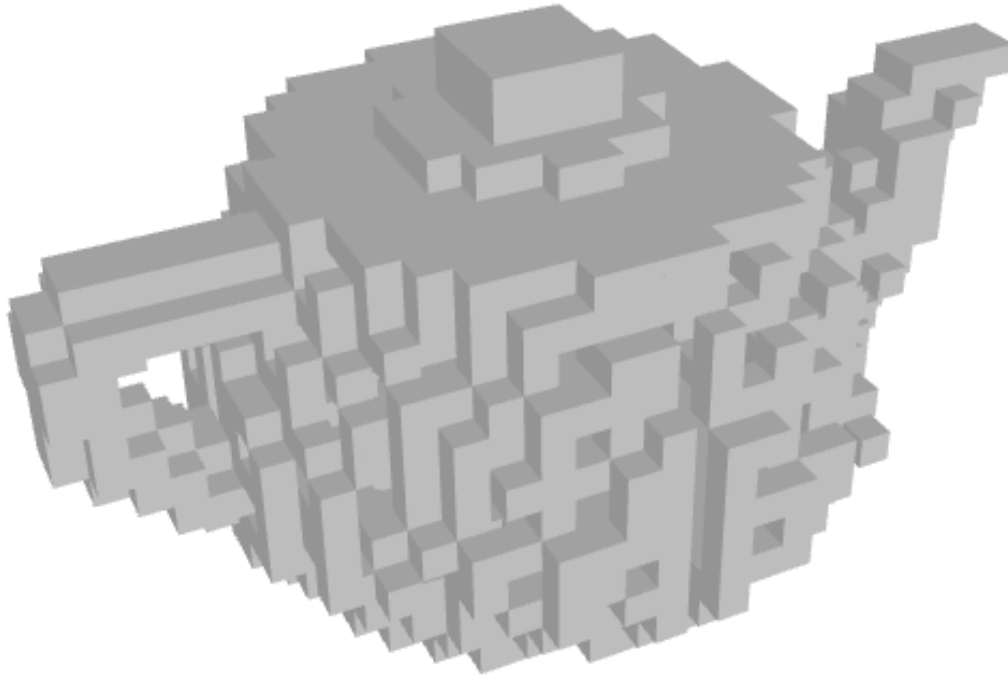


Figure 14: Gambar output voxel teapot pada depth 6

Listing 7: Output terminal teapot pada depth 6

```
$ ./bin/voxelizer.exe test/teapot.obj 6
Berhasil membaca 530 vertex dan 1024 face.
Membangun octree pada kedalaman 6...

Banyaknya voxel yang terbentuk : 1159
Banyaknya vertex yang terbentuk : 9272
Banyaknya faces yang terbentuk : 13908
Kedalaman octree : 6

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 192
5 : 800
6 : 2752
```

Statistik node yang tidak perlu ditelusuri:

```
1 : 0
2 : 0
3 : 40
4 : 92
5 : 456
6 : 1593
```

```
Lama waktu program berjalan      : 0.0774 s
Path file .obj disimpan         : test/teapot-voxelized-6.obj
```

3.8 Hasil 8: Input Tidak Valid

Listing 8: Output terminal input tidak valid

```
\$ ./bin/voxelizer.exe test/teapot.obj abc
Error: kedalamanMaks harus berupa bilangan bulat positif
Penggunaan: ./bin/voxelizer.exe <input.obj> <kedalamanMaks>
    input.obj      : path ke file model 3D
    kedalamanMaks  : kedalaman octree (bilangan bulat >= 1, disarankan 4-8)
```

4 Bonus Visualisasi

Implementasi bonus ini berfokus pada pembuatan *.obj viewer* interaktif dalam bahasa C++. Viewer tersebut menampilkan model 3D hasil voxelization. Implementasi viewer ini berada pada file `src/viewer.cpp` dan `src/viewer.hpp`. Visualisasi digunakan sebagai validasi kualitatif agar bentuk objek, kontur utama, dan distribusi voxel dapat ditinjau langsung dari berbagai sudut pandang.

4.1 Alur Rendering dan Interaksi

4.1.1 Membaca Model dan Menyiapkan Data Geometri

Viewer membaca berkas OBJ hasil voxelization, kemudian memisahkan data menjadi daftar titik (*vertex*) dan daftar segitiga (*face*). Pada tahap ini, struktur geometri disiapkan sebagai data mentah yang akan diproses pada pipeline visualisasi.

4.1.2 Normalisasi Model terhadap *Bounding Box*

Setelah data model terbaca, program menghitung *bounding box* objek untuk mendapatkan pusat model dan ukuran maksimum pada ketiga sumbu. Seluruh titik kemudian ditranslasi ke pusat koordinat dan diskalakan agar dimensi model terjaga dalam rentang tampilan yang stabil. Langkah ini membuat objek dengan ukuran berbeda tetap mudah ditampilkan dan diinteraksikan.

4.1.3 Transformasi Ruang 3D ke Layar 2D

Setiap titik model diputar pada sumbu x dan y sesuai sudut pandang pengguna. Koordinat hasil rotasi lalu diproyeksikan ke koordinat layar 2D melalui fungsi proyeksi, dengan faktor skala yang dipengaruhi oleh nilai *zoom*. Dengan mekanisme ini, perubahan sudut pandang dan pembesaran objek dapat terlihat secara langsung.

4.1.4 Penyaringan, Pengurutan, dan Pewarnaan Segitiga

Untuk setiap segitiga, program menghitung vektor normal permukaan sebagai dasar penentuan visibilitas dan pencahayaan. Segitiga yang membelakangi kamera dipangkas melalui *back-face culling*. Segitiga yang tersisa kemudian diurutkan berdasarkan kedalaman rata-rata (*Painter's Algorithm*) agar tumpang tindih permukaan ditampilkan lebih konsisten. Warna abu-abu tiap segitiga ditentukan dari orientasi normal terhadap arah pandang, sehingga bentuk objek tampak lebih jelas.

4.1.5 Interaksi Pengguna

Interaksi utama yang didukung viewer adalah:

- Klik dan *drag* mouse untuk rotasi objek pada sumbu x dan y .
- *Scroll wheel* untuk *zoom in* dan *zoom out*.

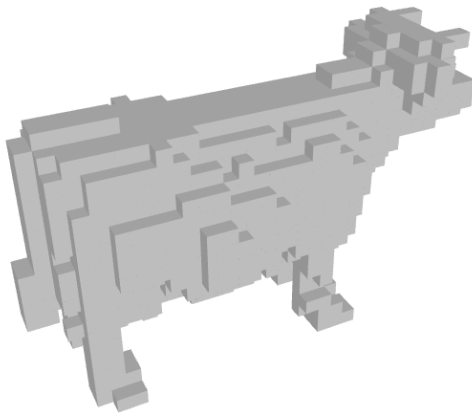
4.2 Perbandingan Visualisasi Web dan Visualizer C++

Subbagian ini menampilkan tiga contoh perbandingan visualisasi antara hasil visualizer C++ dan viewer berbasis web.

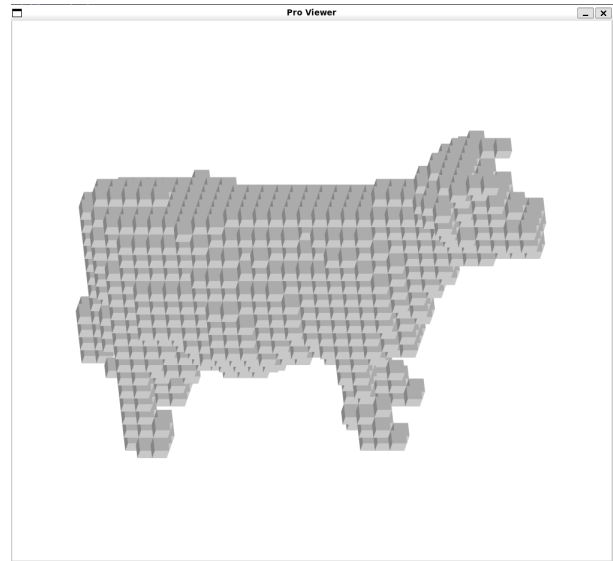
Sumber viewer web yang digunakan:

<https://3dviewer.net/>

4.2.1 Contoh 1: Objek Cow (depth 6)



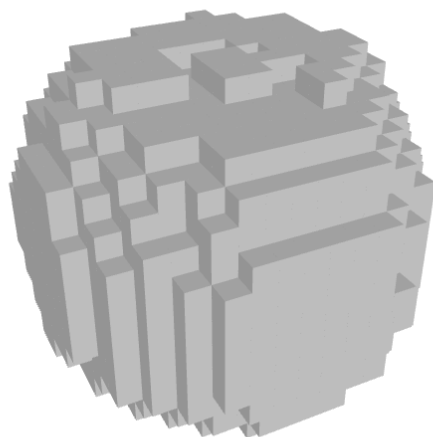
(a) Visualisasi web



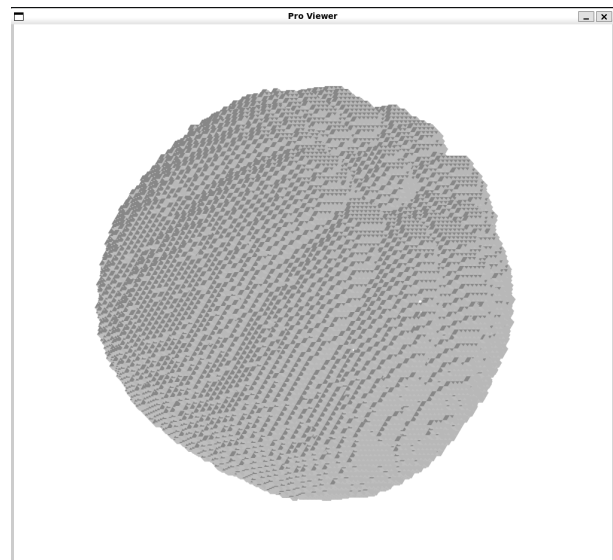
(b) Visualizer C++

Figure 15: Perbandingan contoh 1: visualisasi web dan visualizer C++ pada objek cow depth 6

4.2.2 Contoh 2: Objek Pumpkin (depth 5)



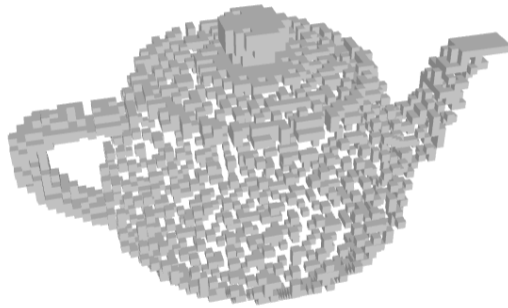
(a) Visualisasi web



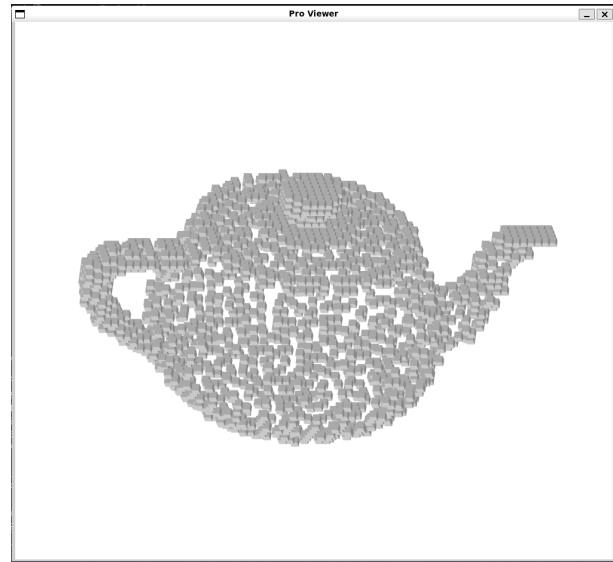
(b) Visualizer C++

Figure 16: Perbandingan contoh 2: visualisasi web dan visualizer C++ pada objek pumpkin depth 5

4.2.3 Contoh 3: Objek Teapot (depth 7)



(a) Visualisasi web



(b) Visualizer C++

Figure 17: Perbandingan contoh 3: visualisasi web dan visualizer C++ pada objek teapot depth 7

4.3 Perbandingan Umum

Secara umum, kedua pendekatan menampilkan bentuk global objek voxel dengan konsisten. Perbedaan utama terlihat pada batas antar-kubus yang saling bersisian. Pada visualisasi web, garis di antara dua *box* yang berdampingan cenderung hilang karena permukaan yang berimpit dirender menyatu. Sebaliknya, pada visualizer C++ yang kami buat, garis pemisah antar-kubus masih terlihat, sehingga struktur voxel diskret tampak lebih tegas.

5 Lampiran

- Tautan Repository:
https://github.com/reinsilitonga/Tucil2_13524090_13524093

6 Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Nashiruddin Akram



Reinsen Silitonga